

ALISE：通过推测式调度加速大型语言模型服务

Youpeng Zhao
University of Central Florida
Orlando, FL, USA
youpeng.zhao@ucf.edu

Jun Wang
University of Central Florida
Orlando, FL, USA
jun.wang@ucf.edu

摘要

大型语言模型（LLM）是通用人工智能（AGI）领域的一项重要进展。以 ChatGPT 为例，基于 LLM 的应用要求推理服务具有尽可能低的响应时延和尽可能高的吞吐量。然而，由于 LLM 执行难以预测，现有服务系统采用的先到先服务（FCFS）策略存在队头（HoL）阻塞与作业响应时间长的问题。

本文提出高效 LLM 推理服务框架 ALISE。其核心设计是利用新的推测式调度器，估计每个作业的执行时间，据此分配适当的优先级顺序，减少异构工作负载的潜在排队延迟。为缓解中间键/值缓存（KV 缓存）的内存开销，我们还采用基于优先级的自适应内存管理和量化压缩。评估表明，相对于先进系统 vLLM，在相同的时延约束下，ALISE 在 Alpaca、ShareGPT 上的推理服务吞吐量最高分别达到 1.8×、2.1×。

ACM 引用格式：

Youpeng Zhao and Jun Wang. 2024. ALISE: 通过推测式调度加速大型语言模型服务. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3676536.3676659>

1 引言

近年来，大型语言模型（LLM）在多种自然语言处理（NLP）任务中展现了广泛能力 [1, 3, 30, 37]。它们无需额外微调即可泛化到多种任务，使下游人工智能应用具有较好的通用性与适应性 [17]。推理服务对于人工智能聊天机器人等 LLM 应用至关重要；交互性要求系统以低时延、高吞吐量生成用户请求的推理结果。

LLM 推理具有独特的自回归特征：根据输入提示 token 与此前生成的全部 token，顺序生成新 token，通常需要运行模型的多次迭代才能完成。通用 DNN 推理服务系统 [5, 10, 23, 36] 侧重图像分类、目标检测等确定且容易预测的工作负载。自回归特性使这些方案难以直接适用于 LLM 并高效扩展，因为每个 token 都依赖此前的 token，顺序依赖导致生成序列的执行时间无法事先确定。

现有 LLM 系统通过定制 CUDA 内核 [6, 7, 16]、迭代级调度 [34, 35]、高效内存管理 [13, 25, 39] 提高服务吞吐量。尽管性能已有很大进展，现有平台仍难以提供低时延，尤其面对真实工作负载时。我们认为，这源于非抢占的先到先服务（FCFS）[38] 策略：异构请求下，它产生队头阻塞。运行至完成的调度会让已调度、可能执行很久的任务阻碍随后较短任务及时处理；这样的排队延迟可能明显降低服务质量（QoS）。如何高效处理

执行时间不同的异构请求，并通过合适调度保持高吞吐量，仍是挑战。

本文提出 ALISE，其名称取自 accelerate LLM serving with a novel speculative scheduler，即用新推测式调度器加速 LLM 服务。关键是估计异构 LLM 请求的执行时间。具体地，采用基于检索的长度预测器：预训练文本编码器先把提示嵌入高维向量，再结合查询数据库与全 MLP 解码器预测输出长度。有了这一先验信息，就能估计各作业执行时间，利用优先级队列进行抢占调度，缓解队头阻塞并减少总响应时间。为管理被抢占作业的中间 KV 缓存，我们设计基于优先级的自适应内存机制，根据各作业的预计等待时间（EWT）动态执行缓存卸载与载入。进一步采用量化，将 KV 缓存压缩到较低精度，减少总体内存开销。

我们在不同服务场景实现并评估 ALISE。广泛评估表明，它在端到端时延和总体吞吐量上优于现有 FCFS 方案。具体而言，相对于先进系统 vLLM [13]，ALISE 在 Alpaca [27]、ShareGPT [28] 上最高分别获得 1.8×、2.1× 的吞吐量。

本文的主要贡献如下：

- 提出基于检索的推测式调度器，估计各新到作业的执行时间，并按优先级调度，降低响应时延。
- 设计自适应内存管理器，智能执行暂不使用的中间 KV 缓存的抢占卸载与载入，并使用量化减少内存开销。
- 在不同场景评估 ALISE，展示其明显优于 ORCA [35]、vLLM [13] 等现有 LLM 推理方案。

本文其余部分安排如下：第 2 节介绍背景与动机，第 3 节阐述系统设计，第 4 节评估，第 5 节回顾相关工作，第 6 节给出结论。

2 背景与动机

Transformer 架构。 Transformer 架构 [31] 推进了自然语言处理，也是大型语言模型的基础。标准 Transformer 包括把 token 序列映射到隐藏维度的嵌入层，以及通过自注意力捕捉输入 token 长期依赖的多层 Transformer。嵌入层将离散单词或 token 转为神经网络可处理的高维向量，表示 token 的语义与位置信息 [15]。每个 Transformer 层主要包括多头注意力（MHA）和逐位置前馈网络（FFN），见图 1(a)。MHA 关注输入不同位置、捕捉上下文，FFN 则执行逐元素变换，引入非线性并提高表示能力。

Transformer 层的核心是注意力模块 [31]，操作见式 1 和 2；为简化说明，省略层归一化与残差连接。涉及查询 Q 、键 K 、值 V 三个中间张量。注意力权重首先由 Q 与 K 的点积计算，再按隐藏维度 d 除以头数 h 后的平方根缩放，最后经过 softmax，即 $\sigma(\cdot)$ 。注意力结果 $\text{Attn}(Q, K, V)$ 由注意力权重乘以 V 得到。

$$\text{Attn}(Q, K, V) = \sigma\left(\frac{QK^T}{\sqrt{d/h}}\right) \cdot V \quad (1)$$

允许免费为个人或课堂使用制作本文全部或部分的电子或纸质副本，条件是不得为盈利或商业利益制作或分发副本，且副本首页保留本声明和完整引用。应尊重本文中由作者之外的主体拥有的组成部分版权。允许注明来源的摘要。其他复制、再出版、发布到服务器或向名单重新分发，需要事先取得明确许可和/或支付费用。许可申请请联系 permissions@acm.org。

ICCAD '24, Oct 27–31, 2024, New Jersey, USA

© 2024 版权所有 / 作者保留版权。Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/24/10

<https://doi.org/10.1145/3676536.3676659>

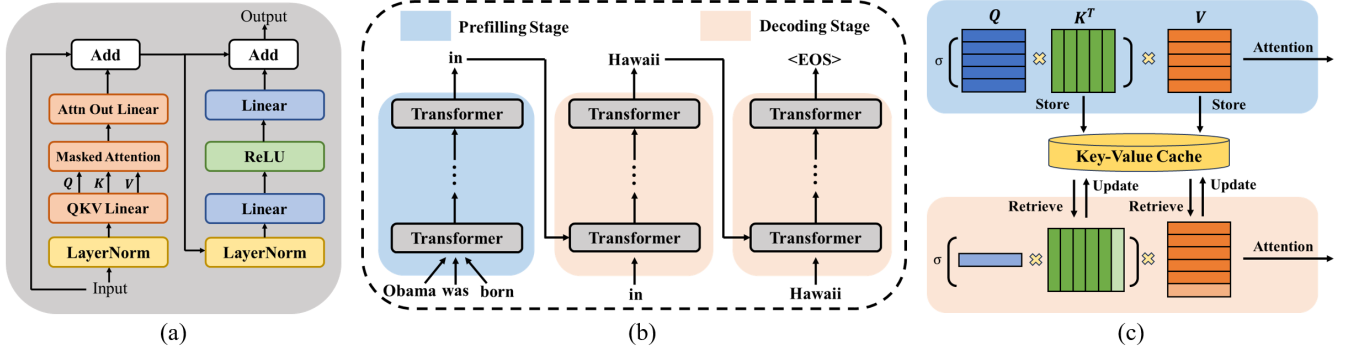


图 1: (a) Transformer 层的操作。(b) 自回归 LLM 推理示例。(c) KV 缓存机制：预填充阶段同时处理全部输入 token，初始化缓存；解码阶段读取已存缓存复用，并逐次更新，直到结束。图内英文：Input、Output 为输入和输出，Add 为残差相加，Linear 为线性层，Attn Out Linear 为注意力输出线性层，Masked Attention 为带掩码注意力，QKV Linear 为查询/键/值线性变换，LayerNorm 为层归一化，ReLU 为激活函数；Prefilling Stage、Decoding Stage 为预填充与解码阶段，Transformer 为模型层；示例“Obama was born in Hawaii”意为“奥巴马出生在夏威夷”，EOS 为终止标记；Key-Value Cache 为键/值缓存，Attention 为注意力，Store、Retrieve、Update 为保存、读取、更新。

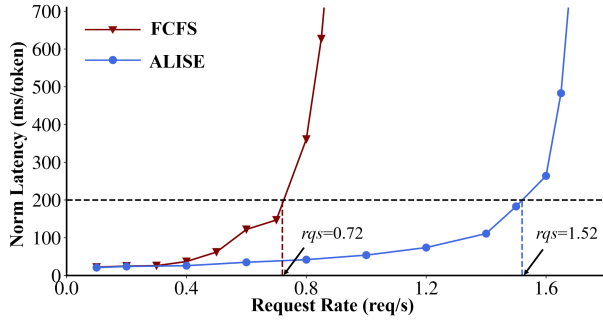


图 2: 在 ShareGPT 上，现有 FCFS 调度与 ALISE 推测式调度的端到端性能比较。图内英文：Norm Latency (ms/token) 为归一化时延（毫秒/token），Request Rate (req/s) 为请求速率（请求/秒）；横向虚线为 200 毫秒/token 约束，竖线标示两种方法可支持的请求速率。

将所有注意力头的输出沿头维度拼接，就得到 MHA 输出；每个头都是一个注意力模块。

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{Attn}_1, \dots, \text{Attn}_h) \quad (2)$$

LLM 推理。生成式推理采用增量解码：先一次计算全部输入提示 token 的激活值，再结合输入与此前生成的所有 token，迭代解码新 token。图 1(b) 给出例子。推理分为预填充与解码两个阶段：预填充一次处理全部输入 token；解码将此生成的输出 token 作为输入，产生下一个 token。因此，解码逐次进行，每次处理一个 token，直到长度达到最大阈值，或输出终止 token “<EOS>”。

自回归特性使系统可以复用中间状态，尤其是 Transformer 层中的键 (K) 和值 (V) 张量，即 KV 缓存 [18, 21]。每个 token 的 KV 缓存依赖此前 token，缓存大小随序列长度线性增长，见图 1(c)。通过 KV 缓存，原先二次复杂度的重复计算被线性复杂度的内存访问替代，以额外 GPU 内存为代价，显著加快推理。

现有 LLM 服务系统的队头阻塞。LLM 的每个 token 依赖此前 token，这种顺序依赖使输出长度难以预测。各请求的执行时间随提示明显变化，同时取决于输入长度和输出长度，即迭代次数。FCFS 中，工作负载一旦被调度，就必须运行到结束；这种运行至完成的调度存在队头阻塞，尤其不适合生成式推理。先调度的任务可能执行很久，阻碍后来的短任务及时运行。

为展示队头阻塞，我们在单张 NVIDIA V100 上运行 OPT-13B [37]，使用 ShareGPT [28] 中不同长度的序列比较 FCFS 与 ALISE。图 2 展示端到端时延。请求速率增加时，FCFS 由于非抢占调度缺乏灵活性，响应时延明显更高。ALISE 利用准确、及时的序列长度与执行时间信息合理调度，提高设备利用率，并在相同时延约束下获得最高 2x 的吞吐量。

3 方法

本文开发新的 LLM 推测式调度机制，并构建 ALISE，应对第 2 节的挑战。图 3 为系统架构。用户提交请求后，基于检索的推测式调度器先预测执行时间，将作业画像送入优先级队列，以迭代为粒度分配优先级，优先处理预计剩余时间最短的作业，缓解队头阻塞。被选作业提交给 GPU 模型执行器，读取相应 KV 缓存计算。ALISE 还包含自适应缓存管理器，在 GPU HBM 与 CPU 内存间动态交换被抢占作业的缓存，并让传输与计算重叠，避免额外时延。

3.1 推测式调度器

输出长度预测。译注（算法 1）：原伪代码在循环内返回，然后才更新数据库；按通常返回语义，更新不会执行。没有相似向量时，MLP 的输入写作检索结果索引，与正文所说输入查询上下文不一致；聚合式没有显式写相似度权重归一化。译稿保留伪代码，不将其自动改写成可执行实现。

支持 LLM 工作负载抢占调度的关键是预测输出长度。已有工作使用较小 LLM [19, 40] 或代理模型 [12, 22] 预测，但通常需要较多工程工作，也产生不可忽略的开销。本文提出基于检索的方法，使用向量数据库 (DB) 以较低内存和时延开销高准确度估计长度；细节见图 4。

先用预训练 BERT 编码器 [8] 将提示转为高维向量，再在向量数据库中进行相似度检索，取相似度高于预设阈值的前 k 个

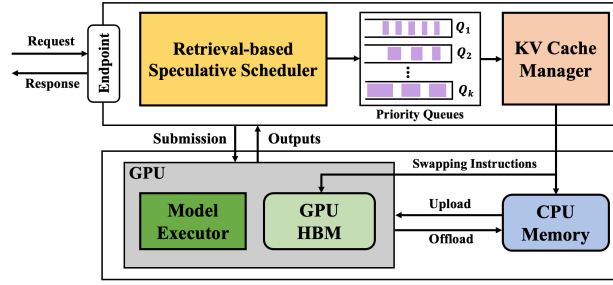


图 3: ALISE 的系统概览。图内英文: Request、Response 为请求和响应, Endpoint 为服务入口, Retrieval-based Speculative Scheduler 为基于检索的推测式调度器, Priority Queues 为优先级队列, KV Cache Manager 为缓存管理器; Submission、Outputs 为提交和输出, Swapping Instructions 为交换指令, Model Executor 为模型执行器, GPU HBM 为 GPU 高带宽内存, CPU Memory 为 CPU 内存, Upload、Offload 为载入与卸载。

历史查询, 使用它们的加权平均预测最终输出长度。若找不到相似查询, 则将查询上下文送入为回归任务微调的全 MLP 解码器。每个请求完成后, 将查询与实际响应长度写回数据库, 保持数据更新。算法 1 给出这一过程。

为评估检索预测, 我们用 OpenChat [32] 构建数据库, 在 Alpaca [27]、ShareGPT [28] 上微调解码器。两个数据集上的平均预测长度误差分别为 3.4%、9.2%; 相对于代理模型方法, 误差改善倍率分别为 1.4×、1.6×。此外, 查询数据库可避免不必要计算, 最多减少 70% 的预测时延。这说明本文推测模型能以较低开销准确预测输出长度。

执行时间预测。给定输出长度后, 可以用数学模型描述推理作业的执行。先把推理分为预填充和解码两部分, 总执行时间表示为:

$$T_{gen}(s, n) = T_{pre}(s) + T_{dec}(s, n) \quad (3)$$

算法 1 基于检索的长度预测

输入: 输入提示 $\{P_j\}_{j=1, \dots, n}$, 相似度阈值 s_0 , 查询数据库 DB , 检索参数 k , BERT 编码器 $BERT_Enc$ 和全 MLP 解码器 MLP_Dec 。

输出: 预测长度 $\{\hat{L}_j\}_{j=1, \dots, n}$

```

1: 当真执行
2:   对所有  $j < n$  执行
3:      $V_j = BERT\_Enc(P_j)$            ▶ 嵌入用户输入提示
4:      $I_j = \arg\max_k S(DB, V_j)$        ▶ 数据库向量检索
5:     如果  $S(I_j) < s_0$  则           ▶ 情形一: 没有相似向量
6:        $\hat{L}_j = MLP\_Dec(I_j)$ 
7:     否则                           ▶ 情形二: 找到相似向量
8:        $\hat{L}_j = \sum S(I_j) \cdot DB(I_j)$    ▶ 聚合检索结果
9:     结束 如果
10:   返回  $\hat{L}_j$ 
11:    $DB.update(V_j, L_j)$            ▶ 更新数据库
12: 结束 对
13: 结束 当

```

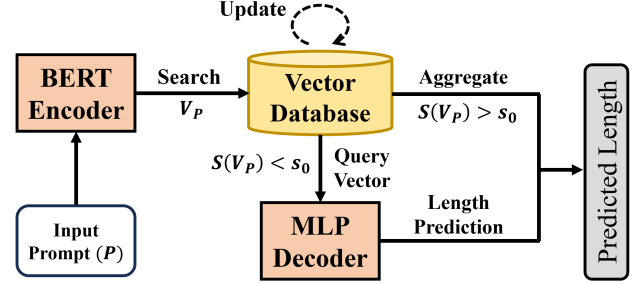


图 4: 基于检索的长度预测器架构。图内英文: Input Prompt 为输入提示, BERT Encoder 为 BERT 编码器, Search 为检索, Vector Database 为向量数据库, Update 为更新, Aggregate 为聚合, Query Vector 为查询向量, MLP Decoder 为 MLP 解码器, Length Prediction、Predicted Length 为长度预测及预测长度; 相似度高于阈值时走检索聚合路径, 低于阈值时走 MLP 路径。

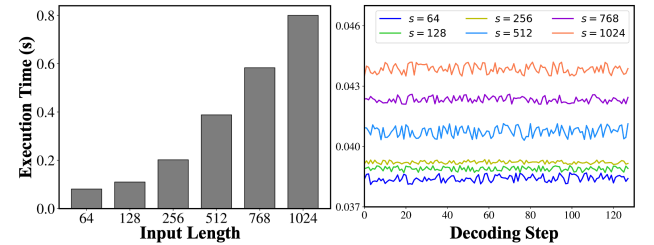


图 5: OPT-13B 的执行时间分解。左图是不同输入长度 (s) 下的预填充时间, 右图是不同解码步骤的时间。图内英文: Execution Time (s) 为执行时间 (秒), Input Length 为输入长度, Decoding Step 为解码步骤; 不同颜色表示不同输入长度。

其中, s 是输入长度, n 是输出长度。在相同硬件与模型下, 预填充时延取决于输入长度, 解码时延同时取决于输入与输出长度。

为研究长度与各阶段时延的关系, 我们在不同输入长度下对 OPT-13B 做推理基准测试。如图 5 所示, 预填充执行时间 $T_{pre}(s)$ 随输入长度 s 近似线性增加。解码时延对输入长度 s 也呈类似线性关系; 由于 KV 缓存, 不同迭代的解码时延 $T_{dec}(s, n)$ 没有明显变化。基于这些观察, 估计预填充与解码时延如下:

$$T_{pre}(s) \approx s \cdot T_0 \quad (4)$$

$$T_{dec}(s, n) \approx n \cdot (\alpha s + \beta) \quad (5)$$

其中, $\{T_0, \alpha, \beta\}$ 是线性回归系数, 可通过对不同输入与输出长度的多组预填充时延样本进行性能剖析来确定。

抢占调度。利用上述分析模型估计 LLM 执行时延, 我们设计了基于优先级队列 [11] 的简单调度器。它维护多个作业队列, 以迭代为粒度, 按预计剩余执行时间分配优先级。对低优先级队列采用虚拟老化 [26] 机制, 防止饥饿。若长度预测偏短、作业超过预测长度, 就将其降低到低优先级队列, 并把预测长度加倍。这样, 推测式调度器可以动态调整优先级, 更有效地管理异构请求。

3.2 自适应内存管理

本文调度器支持迭代级抢占，以管理作业执行顺序，但直接用于 LLM 服务会产生额外内存开销。FCFS 只需保存批次中正在执行作业的 KV 缓存；抢占调度使更多作业在 GPU 上处于等待状态，必须保留缓存供后续使用。KV 缓存大小取决于模型、批大小、序列长度，可能占用大量内存，引发内存不足 (OOM)、停止执行 [25, 39]。

直接的方法是 GPU 内存满时，把新到作业留在优先级队列等待。但若新请求优先级更高、执行更短，却因内存不足无法运行，这种方法就失效。在单 GPU-CPU 等资源受限环境中，推测式调度可能退化为 FCFS，再次出现队头阻塞。另一种方法是删除队列中暂不使用的 KV 缓存，为新作业腾空间，但有两项缺点：第一，低优先级作业重新执行时需重算缓存，增加计算时延；第二，老化机制会将被中止的低优先级作业提升到高优先级，它又中止当前作业，可能导致原文所称的死锁。

动态交换。以往工作利用存储层级加速资源受限环境的 LLM 推理，但多面向离线批量推理，以注意力头或 token 为粒度交换 KV 缓存 [25, 39]。本文设计基于优先级的缓存管理器，以请求为粒度，在 GPU HBM 与 CPU 内存之间动态交换被抢占作业的 KV 缓存。具体地，ALISE 根据预计等待时间 (EWT) 执行交换：EWT 较高的作业倾向于卸载到 CPU DRAM，为高优先级请求腾空间；EWT 较低作业倾向于载入，以备稍后运行。卸载与载入还与计算重叠，避免额外时延。

对优先级为 p 的作业，将优先级更高作业的执行时间求和，得到 EWT：

$$EWT(J) = \sum_{m < p}^m \text{Predictor}(J) \quad (6)$$

由于存在老化机制，老化阈值为 K ，还需比较作业 J_i 升到高优先级队列、开始执行所需时间。因此，将式 6 改写为：¹

$$EWT(J) = \min(\sum_{m < p}^m \text{Predictor}(J), T_{\text{promote}}(J, K)) \quad (7)$$

译注 (算法 2)：原代码的否则分支会把“已在 GPU 且位次仍在上限内”的作业也卸载，与正文保留前若干作业的说明不一致；上限更新写在内层作业循环里，且没有说明外层循环如何重置。边界条件也使用严格小于。此处保留原算法，不据此声称它可直接执行。

利用 EWT，可以设置 GPU 中允许的最大作业数，以决定卸载和载入顺序；这一上限受可用 GPU 内存限制。动态交换细节见算法 2。对每个优先级队列，先计算所有作业的 EWT 并排序 (第 2-3 行)；不超过 GPU 作业上限 M 时，保留被抢占作业，尚未在 GPU 上的缓存则提前载入 (第 5-6 行)；超过上限的作业提前卸载 (第 7-8 行)。每个队列还根据此前队列大小更新上限 M (第 10-11 行)。

KV 压缩。已有工作通过量化模型权重加快注意力计算 [9, 14]。本文量化用于另一目的：压缩暂不使用的 KV 缓存以降低内存开销。我们采用细粒度的按通道量化，以保持效果、减少信息损失。具体地，使用下式将缓存量化为内存中的 b 位整数，再在计算时反量化回原格式；本文原格式为 FP16：

$$x_q = \text{round}(\frac{1}{\lambda}x + z), \quad x = \lambda(x_q - z) \quad (8)$$

¹译注：原文两式的求和上限均写作与索引相同的 m ，求和项也统一写 $\text{Predictor}(J)$ ，没有明确指出各个更高优先级作业的预测剩余时间。其含义应结合正文“对更高优先级作业求和”理解；原式照录，复现时需补充索引定义。

算法 2 动态交换

输入： 优先级作业队列 $\{Q_i\}_{i=1,\dots,h}$ ，GPU 内存 GM ，CPU 内存 CM ，GPU 作业数量上限 M 。

```

1: 当真执行
2:   对  $q \in \{Q_1, \dots, Q_h\}$  执行
3:     EWT( $q$ ).sort()
4:     对  $i = 1$  到  $\text{len}(q)$  执行
5:       如果  $J_i$  不在 GPU 中 且  $i < M$  则
6:         CM.upload( $J_i$ )                                ▶ 提前载入
7:       否则
8:         GM.offload( $J_i$ )                                ▶ 提前卸载
9:       结束 如果
10:      如果  $\text{len}(q) < M$  则                                ▶ 更新 GPU 作业数量上限
11:         $M = M - \text{len}(q)$ 
12:      结束 如果
13:    结束 对
14:  结束 对
15: 结束 当

```

其中，零点为 $z = \lfloor (\frac{-2^b}{\max - \min}) \rfloor$ ，缩放系数为 $\lambda = \frac{\max - \min}{2^b - 1}$ 。本文选择 8 位整数来量化 KV 缓存，尽量减少文本生成质量下降。²

3.3 实现

ALISE 基于 PyTorch [20] 和 HuggingFace Transformers [33] 实现，包含 4.5K 行 Python。执行引擎基于 vLLM [13]，修改以支持迭代级抢占，并与 KV 管理器协作执行动态交换。类似其他 Transformer 服务系统 [13, 16, 35]，我们为层归一化、注意力、ReLU 实现定制融合内核，使用基于 Python 的 Triton 编译器 [29]。为提高 GPU 计算利用率与推理效率，采用迭代级连续组批 [35]，仅对线性层、层归一化、ReLU 等兼容不规则张量的操作组批。

4 评估

4.1 实验设置

硬件。实验采用一个定制高性能 GPU 实例，包含两张显存 32 GB 的 NVIDIA Tesla V100、2.60 GHz Intel Xeon CPU，经 PCIe 4.0×16 连接，并配有 1024 GB DRAM。

工作负载。由于没有公开的 LLM 请求轨迹，我们基于 Alpaca [27] 和 ShareGPT [28] 合成用户请求轨迹。Alpaca 是 GPT-3.5 生成的指令数据集，广泛用于推理评估 [13, 40]；ShareGPT 收集 ChatGPT 用户的聊天对话，内容与序列长度变化更大，见图 7。对数据集进行 token 化，用输入和输出长度模拟真实用户请求；请求时间戳则按泊松分布生成到达时间。

模型。沿用已有服务研究，主实验采用工业界和学术界常用的代表性开源模型家族 OPT [37]，选取 2.7B、6.7B、13B 三种规模。表 1 列出模型及对应 GPU 配置。所有实验中，模型常驻 GPU HBM；模型权重与激活采用 FP16，KV 缓存采用 INT8。

基线。比较以下三个基线：

- ORCA [35]：最早采用迭代级 FCFS 调度的 LLM 服务系统之一。

²译注：原文给出的零点表达式与通常按数值范围和缩放系数确定仿射零点的写法不一致，也没有明确整数裁剪范围。译稿保留原式；复现应针对实现，不能仅靠此式补出完整量化算法。

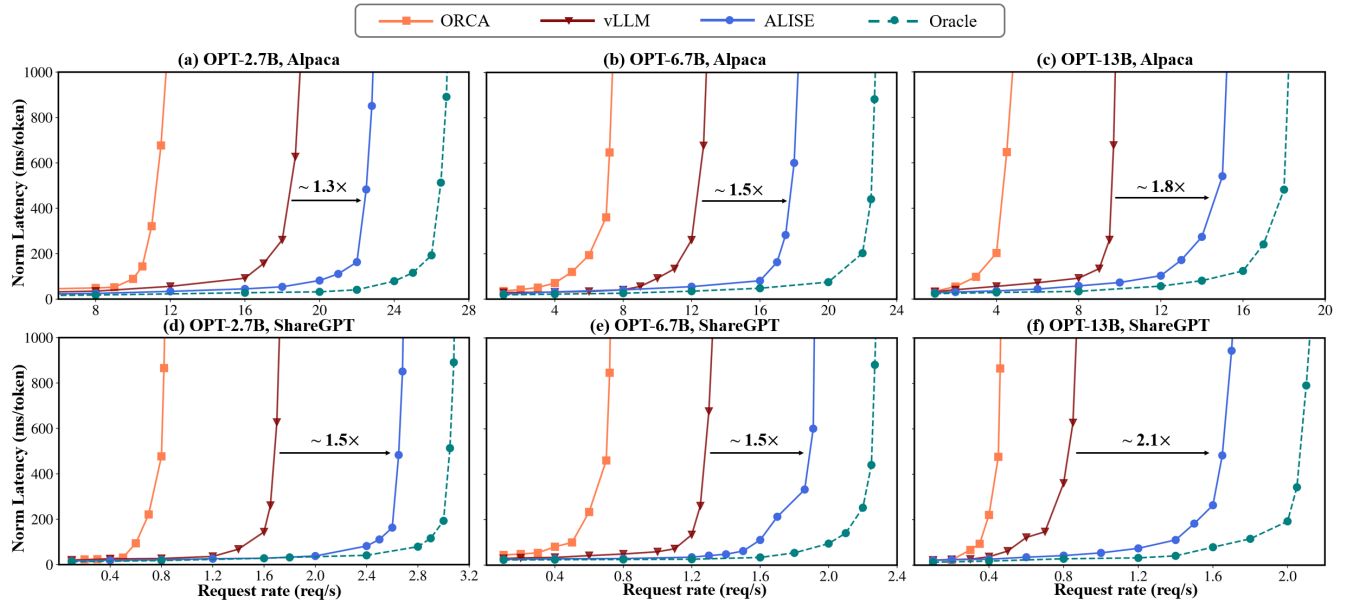


图 6: ALISE 与 ORCA [35]、vLLM [13]、Oracle 在 Alpaca [27] 与 ShareGPT [28] 上的端到端性能比较。图内英文: Norm Latency (ms/token) 为归一化时延 (毫秒 / token), Request rate (req/s) 为请求速率 (请求 / 秒); Oracle 为知道真实输出长度的理想系统, 横向箭头标出相似时延下的请求速率倍率。

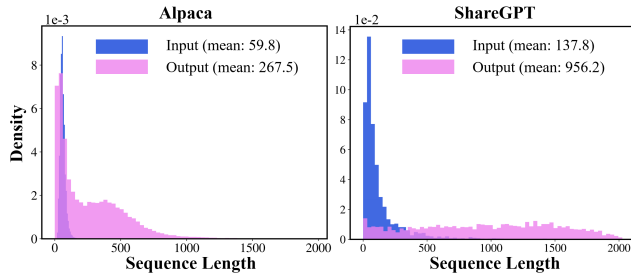


图 7: Alpaca 与 ShareGPT 的输入、输出长度分布。图内英文: Density 为密度, Sequence Length 为序列长度, Input、Output 分别为输入和输出, mean 为均值; Alpaca 输入 / 输出均值为 59.8 / 267.5, ShareGPT 为 137.8 / 956.2。

- vLLM [13]: 先进在线 LLM 推理服务系统, 以块为粒度管理 KV 缓存, 但仍使用 FCFS。
- Oracle: 我们实现具有完美预测器、知道实际输出长度的理想系统, 用作本文性能上界。

指标。使用归一化时延衡量推理服务吞吐能力, 定义为各请求端到端时延除以生成 token 数后取平均。高吞吐量系统应在高请求速率下仍保持低归一化时延。批大小按 GPU 内存容量尽可能增大, 所有系统均使用 30 分钟轨迹评估。

4.2 端到端性能

图 6 第一行展示 Alpaca 结果: 请求速率增大时, 归一化时延逐渐增加, 超过阈值后急剧上升。这与已有工作 [13, 25, 35, 39] 一致: LLM 推理通常受内存限制, 到达负载超过系统承载能力时, 请求池持续增长, 响应时延不断上升。在 Alpaca 上, ALISE 在相近时延下达到 vLLM 的 1.3 ~ 1.8× 吞吐量, 源于推测式调度

表 1: 模型配置。

模型	头数	层数	隐藏维度	参数大小
OPT-2.7B	32	32	2560	5 GB
OPT-6.7B	40	40	5120	13 GB
OPT-13B	40	40	5120	24 GB

动态调整优先级、降低响应延迟。例如, 图 6(b) 在原文所称 200 ms 时延约束下, ALISE 处理的请求数为 vLLM 的 1.5×; 相对于 ORCA, 最高支持 3.1× 的请求速率。图 6 第二行的 ShareGPT 结果显示优势更明显, 因为平均输入与输出更长, 波动也大得多。相对于 vLLM, ALISE 最高达到 2.1× 吞吐量; 相对于 ORCA, 在相近时延约束下最高达到 4.3×。³

4.3 性能分析

KV 缓存管理的影响。为展示自适应缓存管理的收益, 我们与两个简单基线比较: 重算和延后。重算策略删除被抢占低优先级作业的缓存, 重新调度时再计算; 延后策略则在 GPU 内存不足时, 直接推迟新到作业。

在 Alpaca 上, 以不同请求速率运行 OPT 模型。如图 8 所示, 速率较低时, 多数作业的缓存都在 GPU, ALISE 与两基线差别不大。速率上升后, 差距增大: 在相同请求速率下, 延后与重算的归一化时延最多分别为 ALISE 的 1.8×、2.8×。延后策略在高请求速率时又出现队头阻塞, 更多执行较短的新请求被推迟, 响应时延上升; 重算策略的额外时延主要来自 token 生成

³译注: 图的纵轴单位为毫秒 / token, 正文前面也定义了归一化时延。因此此处 200 ms 的图示约束不能直接引用为单个请求端到端总时延 200 ms; 吞吐量倍率来自相近归一化时延下可支持的请求速率。

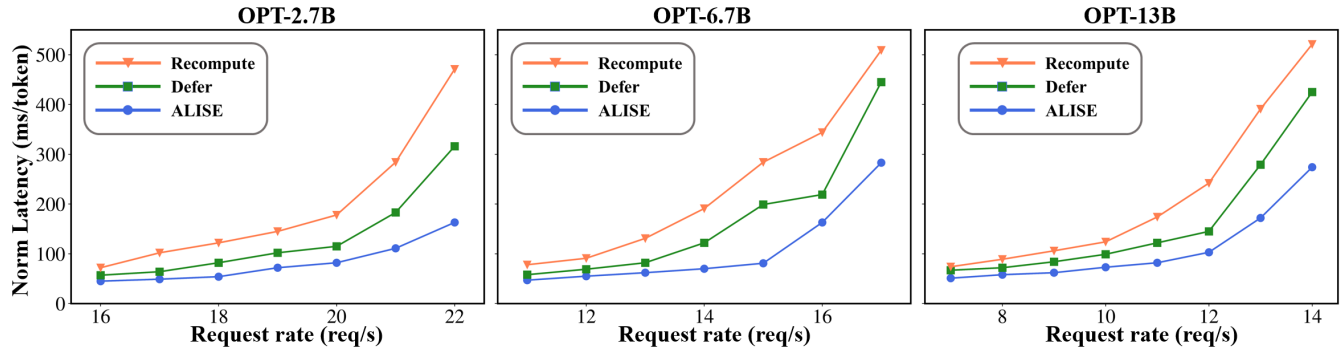


图 8: Alpaca 上, ALISE 不同内存管理策略的时延比较。图内英文: Request rate (req/s) 为请求速率 (请求/秒), Norm Latency (ms/token) 为归一化时延 (毫秒/token); Recompute 为重算策略, Defer 为延后新请求策略, ALISE 为动态交换策略。

表 2: 在 ShareGPT 上, 本文基于检索的长度预测器的准确率与吞吐量评估。

指标	OPT-2.7B		OPT-6.7B		OPT-13B	
	代理模型	检索方法	代理模型	检索方法	代理模型	检索方法
准确率 ($\uparrow$)	0.781	0.821	0.712	0.856	0.634	0.744
预测误差 ($\downarrow$)	0.122	0.057	0.145	0.096	0.178	0.123
平均预测时延 ($\downarrow$)	12.2 ms	3.92 ms	11.7 ms	4.74 ms	14.8 ms	4.49 ms
吞吐量 ($\uparrow$)	1×	1.47×	1×	1.63×	1×	1.82×

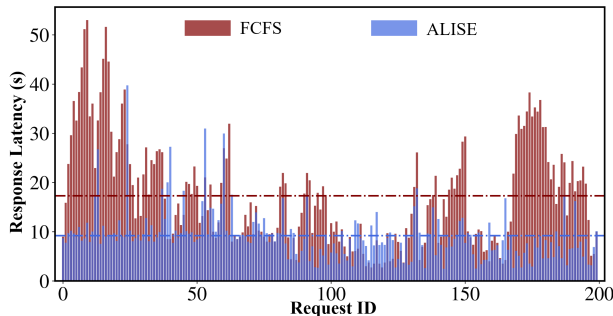


图 9: 在 ShareGPT 上以每秒 2 个请求服务 OPT-13B 时, 采样 200 个请求的响应时延; 虚线表示平均响应时延。图内英文: Response Latency (s) 为响应时延 (秒), Request ID 为请求编号, FCFS 与 ALISE 为两种调度方法。

时间增加, 因为被抢占作业恢复执行时, 必须重算此前迭代的 KV 缓存, 才能生成后续 token。

长度预测器的影响。我们在 ShareGPT 上, 将检索预测器与已有代理模型预测器 [12, 22] 比较准确率与吞吐量。衡量准确率时, 把长度等宽分桶; 预测长度与实际长度落在同一桶, 就视为准确。表 2 表明, 本文在两方面均更好。历史用户查询数据库使新查询能以较低误差准确预测; 一些查询无需经过预测模型, 还显著降低了预测时延。长度信息更准确后, 调度器可以分配更适当的优先级, 提高总体推理吞吐量。

响应时延分析。为理解推测式调度对系统吞吐量的影响, 我们在 ShareGPT 上服务 OPT-13B 时, 随机采样连续生成的 200 个请求, 比较非抢占 FCFS 与 ALISE 的逐请求端到端时延。如

表 3: ShareGPT 与 Alpaca 上不同模型的吞吐量提升; 原文单位为每秒处理的请求数。

	ORCA	vLLM	ALISE
Alpaca (30 请求/秒)			
LLaMA-13B	42.23	71.87	101.42 (+41%)
LLaMA-7B	75.42	122.87	164.51 (+34%)
Pythia-12B	34.85	64.19	91.12 (+42%)
ShareGPT (2 请求/秒)			
LLaMA-13B	14.42	27.89	41.22 (+47%)
LLaMA-7B	30.28	62.93	87.32 (+39%)
Pythia-12B	12.36	24.95	37.27 (+49%)

图 9 所示, 抢占缓解了队头阻塞, 大多数请求的响应时延明显下降; 相对于 FCFS, 平均响应时延降低接近 46%。

更多模型上的结果。为验证跨模型效果, 我们进一步选取 LLaMA-7B/13B [30]、Pythia-12B [2] 等常见开源 LLM。表 3 表明, ALISE 在这些模型上也持续优于基线。

译注 (表 2): 原文称表中数值单位为请求/秒, 但多项数值明显高于表头给定的到达速率, 如每秒到达 2 个请求时报告处理数十个请求。若为同一稳定运行窗口的已完成请求数, 二者无法同时成立。原文没有说明不同计时窗口或其他计量单位; 这里保留表格、单位与数值, 不能自行改称 token/秒。

5 相关工作

DNN 推理服务。近年来，人工智能应用在现实中迅速增长，促使研究者开发专门系统 [5, 10, 23, 36]。这些系统通常为 ResNet 等较小模型优化组批 [5]、调度 [10]、缓存 [36]。INFaaS [23] 进一步通过在模型变体与性能之间选择最佳权衡，自动化模型选择和部署。然而，它们没有考虑 LLM 推理的多迭代特点与不可预测性，错过了进一步优化的机会。

LLM 推理服务。已有专门服务系统面向基于 Transformer 的 LLM [13, 16, 35]。FasterTransformer 通过 GPU 内核优化和模型并行，较早优化了大型 Transformer 的训练与推理服务 [16]；ORCA 利用多迭代特性，提出迭代级调度与选择性组批 [35]；vLLM 通过 PagedAttention 将 KV 缓存存于非连续分页内存，缓解碎片 [13]。尽管效果较好，现有系统仍采用非抢占 FCFS [38]。运行至完成的调度一旦开始就必须执行到结束，长输出作业阻塞随后的短作业，导致队列阻塞、增加响应时延。真实请求具有异构性，缺乏灵活性的 FCFS 成为低时延、高吞吐量服务的新瓶颈。

LLM 输出长度预测。预测 LLM 输出长度的工作仍较少。S³ 使用 DistilBERT [24] 根据输入查询分类输出长度范围；SSJF [22] 用类似代理模型进行分类与回归。也有工作通过指令微调，让 LLM 自身输出预计长度，但通常引入额外偏差、需要较多工程工作 [19, 40]。本文采用向量数据库上的检索预测，以低时延开销提供更准确长度信息。

LLM 内存优化。已有多种技术减少 LLM 推理内存开销。Sparse-Transformer 对序列 token 生成固定步长的稀疏模式，加快推理 [4]；FlashAttention 通过内核级细粒度分块与划分，减少 GPU 片上 SRAM 和片外 HBM 间的访问 [6, 7]；FlexGen 面向资源受限单 GPU-CPU 场景，借助 CPU DRAM 与二级存储，对 KV 缓存、权重及中间结果静态卸载 [25]；ALISA 通过算法与系统协同设计，利用推理稀疏模式，在缓存与重算间取得最佳权衡 [39]。本文与这些内存优化技术互补，关注调度，以降低在线异构工作负载的响应时延。

6 结论

本文提出高效 LLM 推理服务系统 ALISE，解决现有 FCFS 调度的队列阻塞。ALISE 用基于检索的推测式调度器，按剩余执行时间抢占调度，减少异构工作负载的排队延迟。为降低被抢占作业的内存开销，设计自适应内存管理，动态交换暂不使用的 KV 缓存，并使用量化压缩。实验表明，相对于 vLLM 等先进系统，在 Alpaca、ShareGPT 上，ALISE 最高分别获得 1.8×、2.1× 吞吐量。

致谢

本工作部分获得美国国家科学基金会（NSF）项目 1907765、2400014 资助。感谢 ICCAD 匿名审稿人的建设性意见，帮助改进本文。

参考文献

- [1] Meta AI. 2024. Introducing Llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>
- [2] Stella Rose Biderman, Hailey Schoelkopf, Quentin G. Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shrivanshu Purohit, Usvsn Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Matusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *ArXiv abs/2005.14165* (2020).
- [4] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *ArXiv abs/1904.10509* (2019).
- [5] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 613–627.
- [6] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *ArXiv abs/2307.08691* (2023).
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *ArXiv abs/1810.04805* (2019).
- [9] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *International Conference on Learning Representations (ICLR)* (2023).
- [10] Arpan Gujarati, Reza Karimi, Safya Alzayat, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *USENIX Symposium on Operating Systems Design and Implementation*.
- [11] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. 2019. Priority queues. *The Art of Multiprocessor Programming* (2019).
- [12] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S3: Increasing GPU Utilization during Generative Inference for Higher Throughput. *Advances in Neural Information Processing Systems* 36 (2023).
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Huang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Haotian Zhang, and I. Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles* (2023), 611–626.
- [14] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *ArXiv abs/2306.00978* (2023).
- [15] Tomas Mikolov, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *International Conference on Learning Representations*.
- [16] NVIDIA. 2019. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>
- [17] OpenAI. 2022. Introducing ChatGPT. <https://openai.com/blog/chatgpt>
- [18] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A Fast, Extensible Toolkit for Sequence Modeling. In *North American Chapter of the Association for Computational Linguistics*. 6151–6162.
- [19] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. 2022. Training language models to follow instructions with human feedback. *ArXiv abs/2203.02155* (2022).
- [20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [21] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shrivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023).
- [22] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishanker K Iyer. 2024. Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction. *ArXiv abs/2404.08509* (2024).
- [23] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. 2021. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 397–411.
- [24] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *ArXiv abs/1910.01108* (2019).
- [25] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark W. Barrett, Joseph Gonzalez, Percy Liang, Christopher Ré, Ioan Cristian Stoica, and Ce Zhang. 2023. High-throughput Generative Inference of Large Language Models with a Single GPU. In *International*

- Conference on Machine Learning*. PMLR, 31094–31116.
- [26] Abraham Silberschatz, Greg Gagne, and Peter Galvin. 2004. *Operating System Principles*.
- [27] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [28] ShareGPT Team. 2023. ShareGPT. <https://sharegpt.com/>
- [29] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (2019).
- [30] Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *ArXiv abs/2307.09288* (2023).
- [31] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. *ArXiv abs/1706.03762* (2017).
- [32] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. OpenChat: Advancing Open-source Language Models with Mixed-Quality Data. *ArXiv abs/2309.11235* (2023).
- [33] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2019. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *ArXiv abs/1910.03771* (2019).
- [34] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).
- [35] Gyeong-In Yu and Joo Seong Jeong. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX Symposium on Operating Systems Design and Implementation*.
- [36] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In *Symposium on Networked Systems Design and Implementation*.
- [37] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *ArXiv abs/2205.01068* (2022).
- [38] Wei Zhao and John A. Stankovic. 1989. Performance analysis of FCFS and improved FCFS scheduling algorithms for dynamic real-time computer systems. *[1989] Proceedings. Real-Time Systems Symposium* (1989), 156–165.
- [39] Youpeng Zhao, Di Wu, and Jun Wang. 2024. ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching. *ArXiv abs/2403.17312* (2024).
- [40] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline. *ArXiv abs/2305.13144* (2023).